

数据库系统 Database System

Instructor: Yiwen Shen, Ph.D.

Dept. of Software & Computer Engineering,

Ajou University, Korea

chrisshen@ajou.ac.kr

今日提纲

- 复习
 - 中级 SQL
 - 高级 SQL
- 使用 ER 模型的数据库设计

使用**ER** 模型 的数据库设计

Entity-Relationship Model

大纲

- 设计过程概述
- 实体关系模型
- 复杂属性
- 映射基数
- 主键
- 删除实体集中的冗余属性
- 将 ER 图简化为关系模式
- 扩展 ER 功能
- 实体关系设计问题
- 数据建模的替代符号
- 数据库设计的其他方面

设计阶段

- 第一阶段 – 表征出潜在数据库用户的数据需求。
- 第二阶段 – 选择数据模型
 - 应用所选数据模型的概念
 - 将这些要求转化为数据库的概念化模式。
 - 一个完全开发的概念化的模式表明了企业的功能需求。
 - 描述将对数据执行的操作（或事务）类型。

设计阶段 (续)

- 最后阶段 -- 从抽象数据模型转向数据库的实现
 - 逻辑设计 -- 决定数据库模式。
 - 数据库设计要求我们找到一个“好的”关系模式集合。例如：
 - 基于商业决策 -- 我们应该在数据库中记录哪些属性？
 - 基于计算机科学 (i.e., 技术角度) 决策 -- 我们应该拥有什么样的关系模式，以及属性应该如何在各种关系模式之间分布？
 - 物理设计 -- 决定数据库的物理布局

设计方案

- 在设计数据库模式时，我们必须确保避免**两个主要陷阱**：
 - **冗余**：糟糕的设计可能会导致**重复的信息**。
 - 信息的冗余表示可能导致信息的各个副本之间的**数据不一致**
 - **不完整性**：糟糕的设计可能会使企业的某些方面难以或无法建模。
- 仅仅避免糟糕的设计是不够的。
 - 可能有大量好的设计可供我们选择。

设计方法

- **实体关系模型 (Entity Relationship Model)**
 - 将企业建模为 **实体 (Entity)** 和 **关系 (Relationship)** 的集合
 - **实体**: 企业中可与其他对象区分的“事物”或“对象”
 - 由一组**属性描述**
 - **关系**: 多个实体之间的关联
 - 以 **实体关系图 (an entity-relationship diagram)** 的形式表示:
- **规范化理论 (Normalization Theory)**
 - 明确哪些设计不好, 并进行测试

ER模型 - 数据库建模

- ER数据模型的开发是为了通过允许指定代表数据库整体逻辑结构的企业模式来促进数据库设计。
- ER 数据模型采用三个基本概念：
 - 实体集 (entity sets),
 - 关系集 (relationship sets),
 - 属性 (attributes)。
- ER模型还有一个相关的图表表示，即ER图，它可以图形化地表达数据库的整体逻辑结构。

实体集 (Entity Sets)

- **实体**是存在并且可与其他对象**区别**开的对象。
 - 例如：特定人员、公司、事件、工厂
- **实体集**是具有相同属性的同一类型的实体的集合。
 - 例如：所有人、公司、树木、假期的集合
- 实体由**一组属性表示**；即实体集的所有成员所具有的描述性属性。
 - 例子：

instructor = (ID, name, salary)

course = (course_id, title, credits)

- 属性的子集形成实体集的**主键**；即唯一地标识集合的每个成员。

实体集-教师(*instructor*) 和 学生(*student*)

76766	Crick
45565	Katz
10101	Srinivasan
98345	Kim
76543	Singh
22222	Einstein

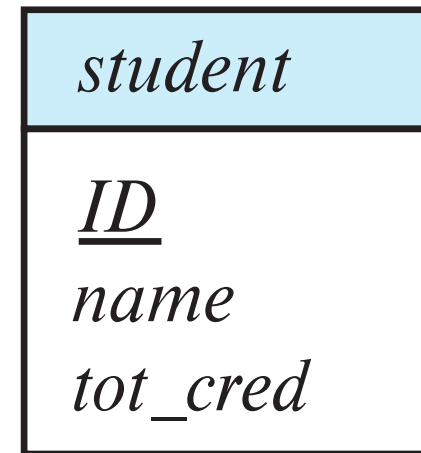
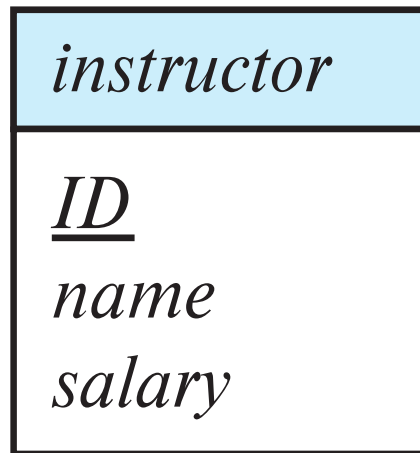
instructor

98988	Tanaka
12345	Shankar
00128	Zhang
76543	Brown
76653	Aoi
23121	Chavez
44553	Peltier

student

在 ER 图中表示实体集

- 实体集可以用图形表示如下：
 - 矩形代表实体集。
 - 在实体矩形里面 属性 被列出
 - 下划线表示主键属性



关系集 Relationship Sets

- 关系是多个实体之间的关联

例子:

44553 (Peltier) advisor 22222 (Einstein)
student entity *relationship set* *instructor* entity

- 关系集 是 $n \geq 2$ 个实体之间的数学关系，每个元素都取自实体集

$$\{(e_1, e_2, \dots, e_n) \mid e_1 \in E_1, e_2 \in E_2, \dots, e_n \in E_n\}$$

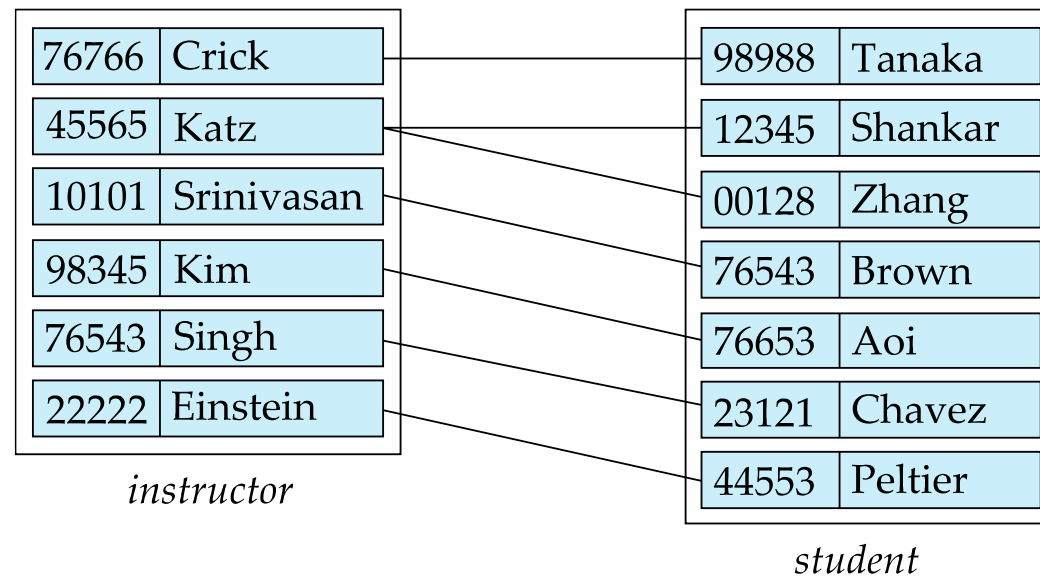
其中 $(e_1, e_2, \dots, e_n)$ 是关系实例

- 例子:

$$(44553, 22222) \in \text{advisor}$$

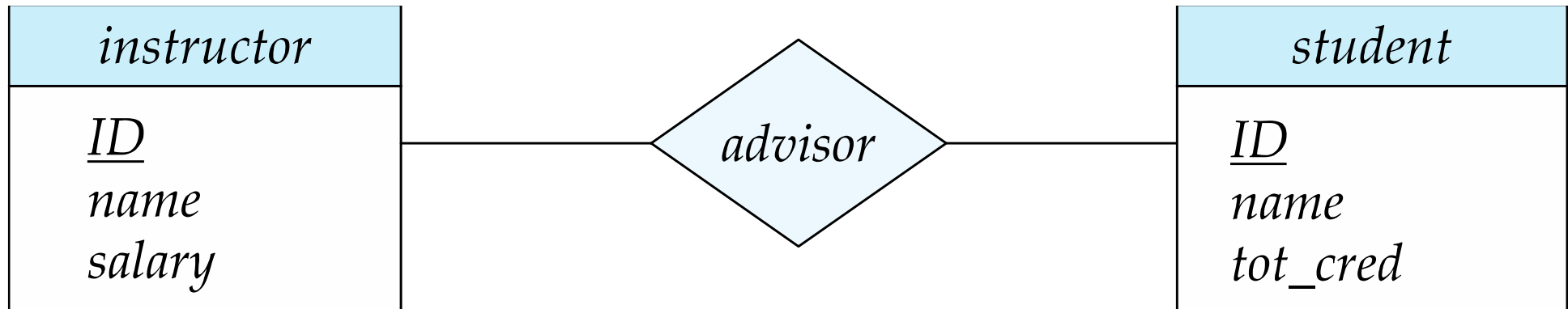
关系集 (续)

- 例如：我们定义关系集 *advisor* 来表示学生和作为其导师的老师之间的关联。
- 以图形方式，我们在相关实体之间画一条线。



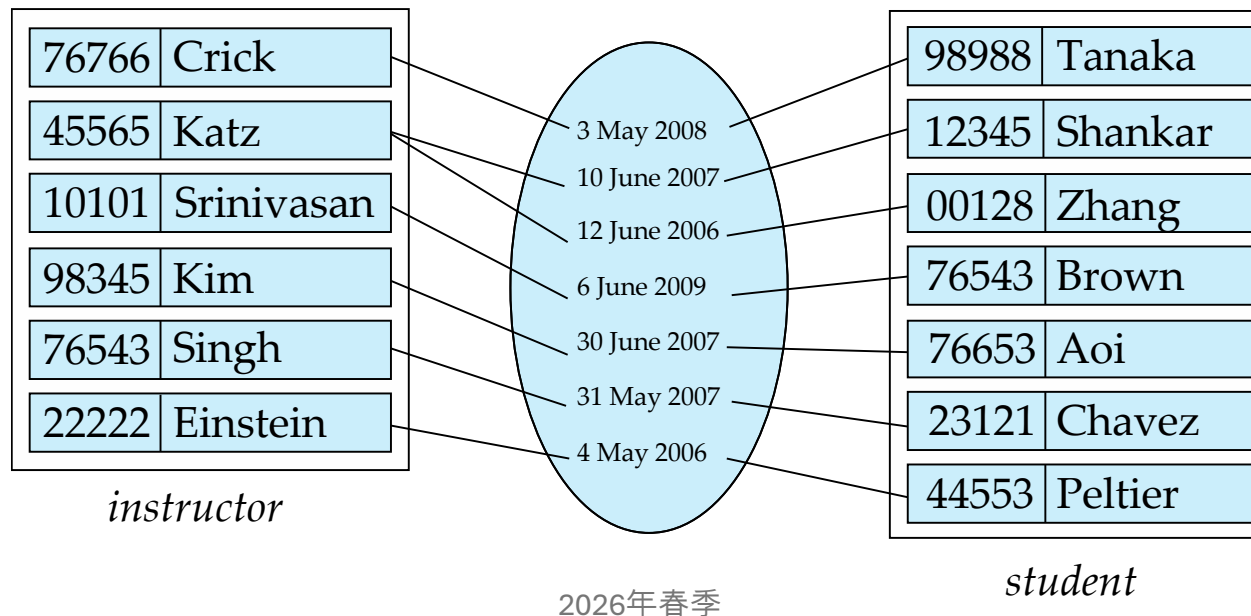
通过 ER 图表示关系集

- 菱形 (Diamonds) 代表关系集。

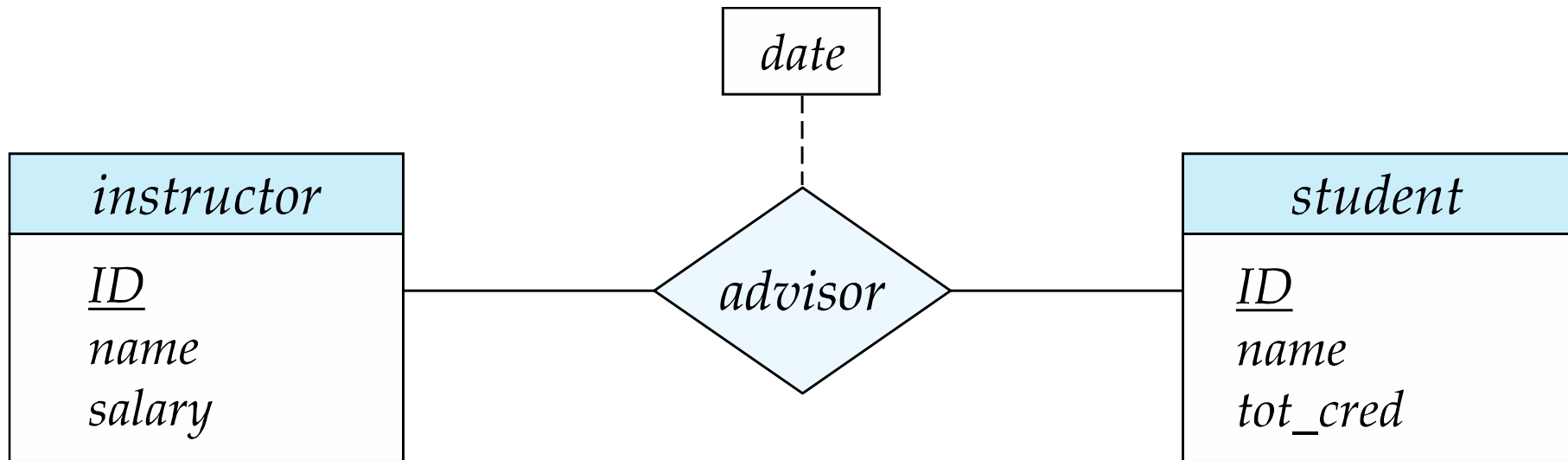


关系集 (续)

- 属性 (attribute) 也可以与关系集相关联。
- 例如，实体集 *Instructor* 和 *Student* 之间的 **Advisor** 关系集 可能具有属性 **date**，用于跟踪学生何时开始与 **Advisor** 建立联系

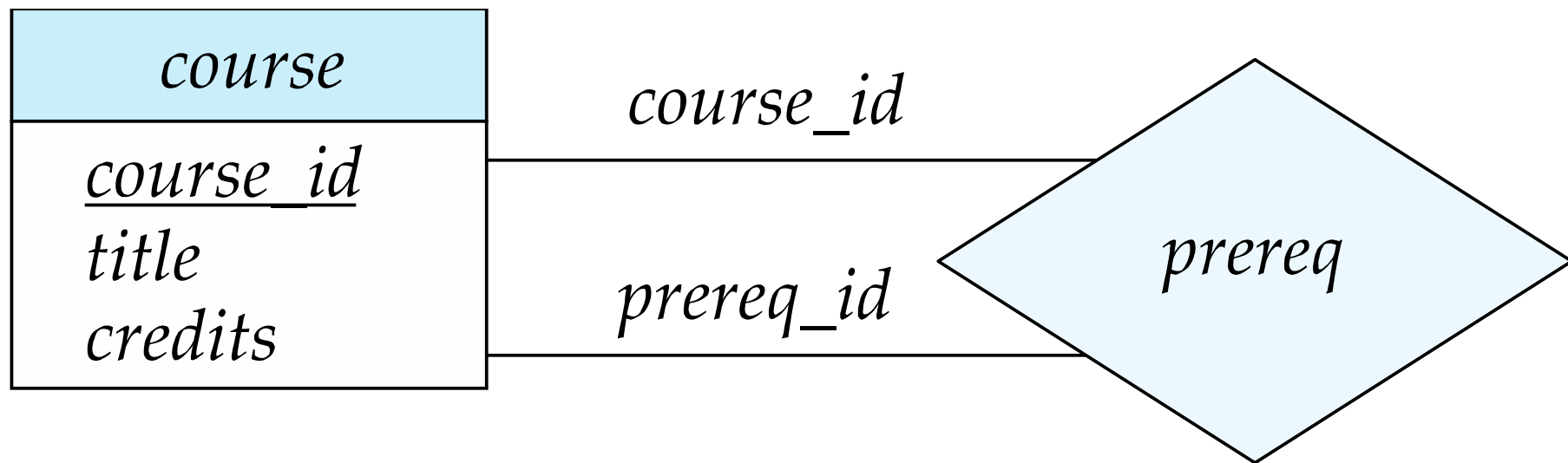


具有属性的关系集



角色 Roles

- 关系的实体集**不需要是独特的 (distinct)**
 - 在关系中, 实体集的每次出现都扮演着一个“**角色**”
- 标签“*course_id*”和“*prereq_id*”称为**角色**。

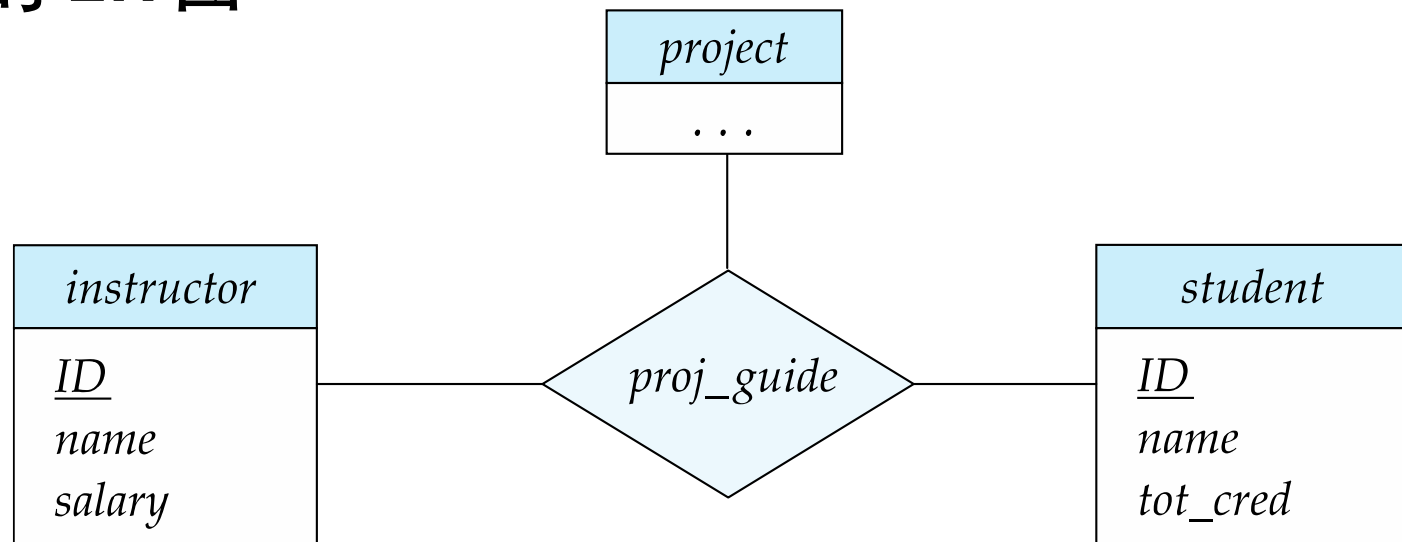


关系集的度 (Degree)

- **二元**关系
 - 涉及两个实体集（或二度）。
 - 数据库系统中的大多数关系集都是**二进制的**。
- 两个以上实体集之间的关系很少见。大多数关系都是**二元的**。
 - 例如：学生在 导师 (*instructor*) 的指导下进行 研究 *项目*。
 - 关系 **项目指导** (*proj_guide*) 是 导师、学生和 项目 之间的三元关系 (a ternary relationship)

非二元关系集

- 大多数关系集都是二元的
- 有时候，将关系表示为非二元关系会更方便。
- 三元关系的 ER 图



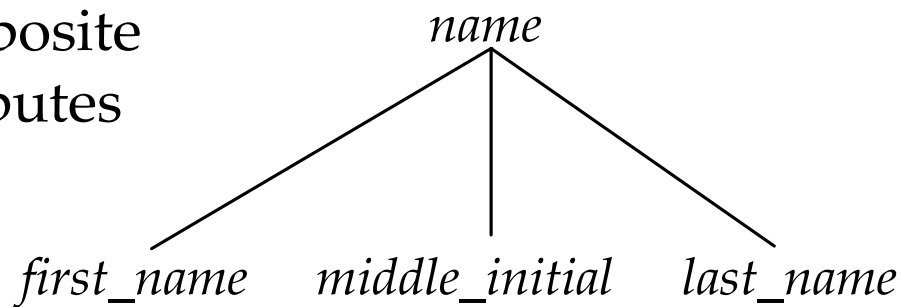
复杂属性

- 属性类型：
 - 简单属性和复合属性。
 - 单值属性和多值属性
 - 示例：多值属性： `phone_numbers`
 - 派生属性
 - 可以根据其他属性计算
 - 例如：年龄，给定出生日期
- 域- 每个属性的允许值集合

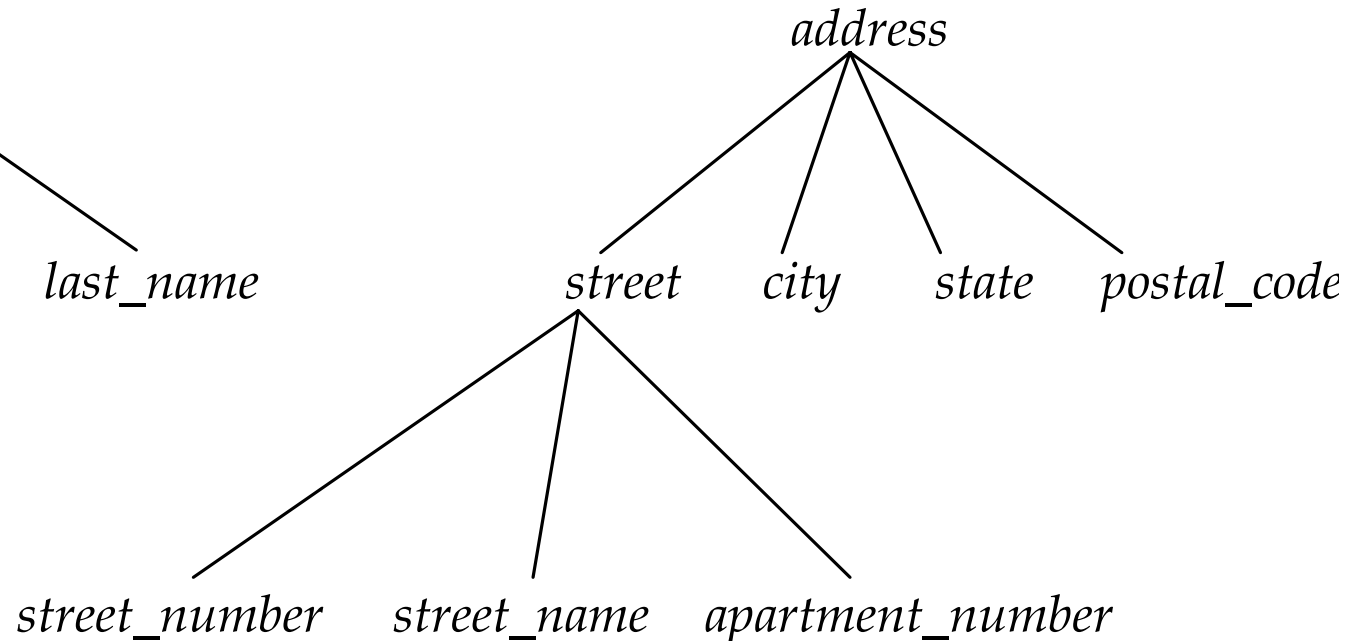
复合属性

- **复合属性**允许我们将属性划分为子部分（其他属性）。

composite
attributes



component
attributes



在 ER 图中表示复杂属性

- 具有复合、多值和派生属性的 ER 图。

instructor

ID

name

first_name

middle_initial

last_name

address

street

street_number

street_name

apt_number

city

state

zip

{ *phone_number* }

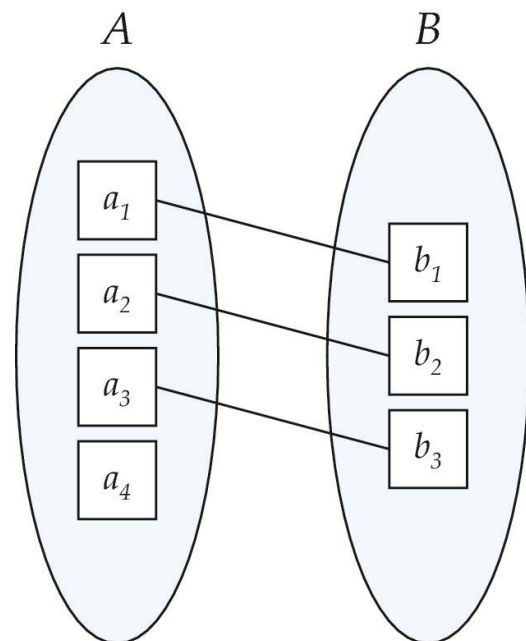
date_of_birth

age ()

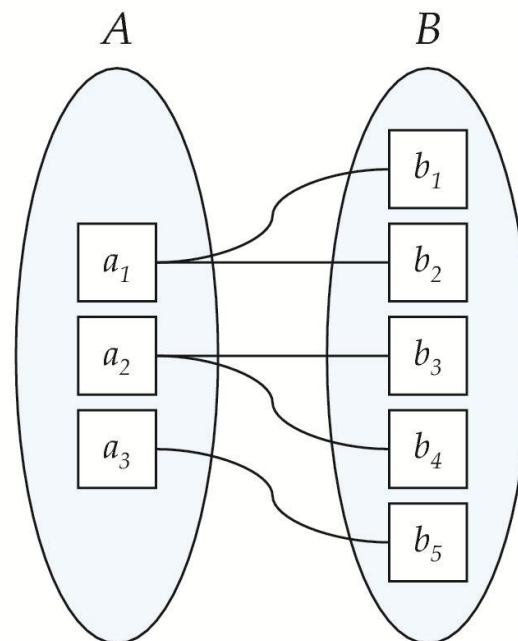
映射基数 (Mapping Cardinality) 约束

- 表达可以通过关系集与另一个实体关联的**实体的数量**。
- 在描述二元关系集时最有用。
- 对于二元关系集，**映射基数**必须是以下类型之一：
 - 一对一
 - 一对多
 - 多对一
 - 多对多

映射基数



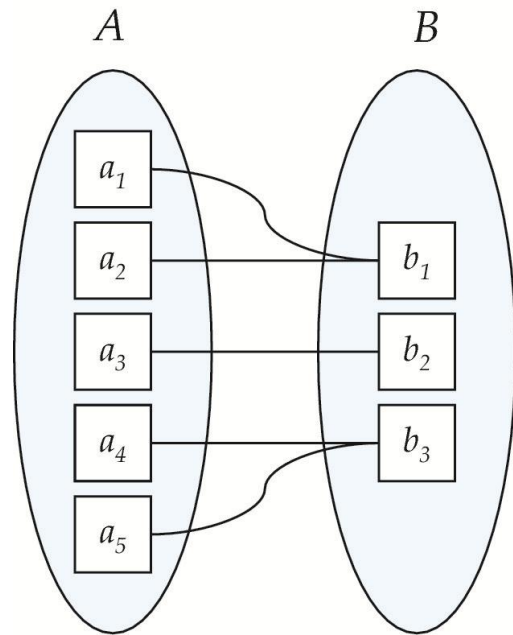
(a)
一对一



(b)
一对多

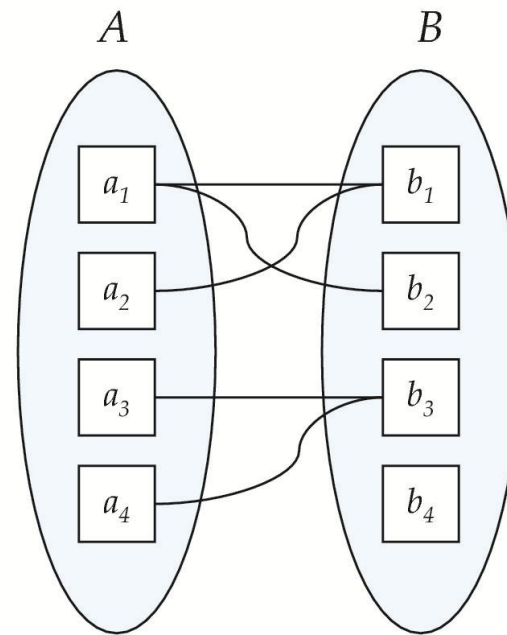
注意：A和B中的某些元素可能无法映射到在另一个集合中任何的元素

映射基数



(a)

多对一



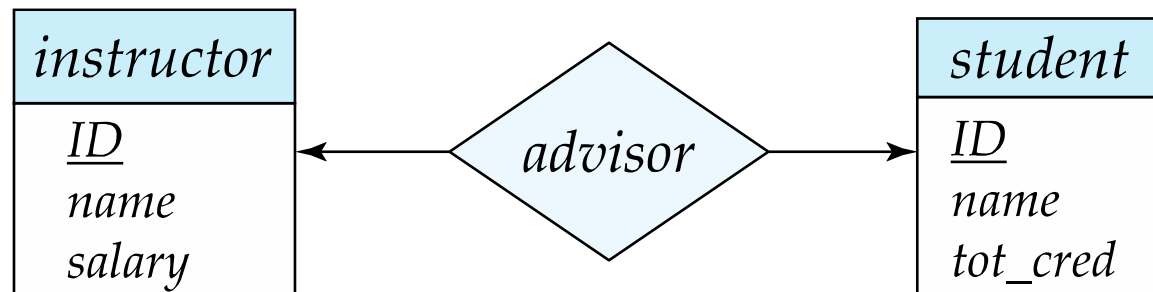
(b)

多对多

注意: A 和 B 中的某些元素可能无法映射到任何另一个集合中的元素

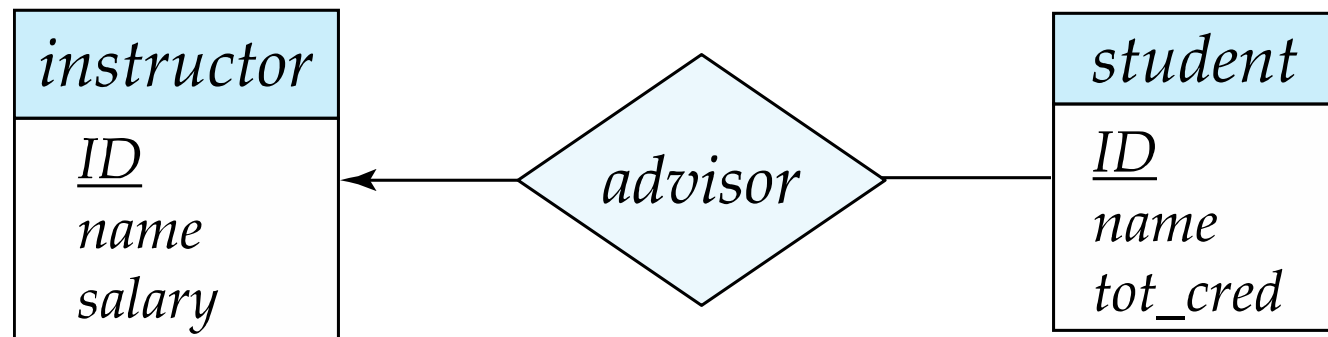
在 ER 图中表示基数约束

- 在关系集和实体集之间绘制有向线（→），表示“一 (one)”或无向线（—），表示“多 (many)”来表达基数约束。
- 教员 (instructor) 和学生 (student) 之间的一对一关系：
 - 一名学生最多通过关系 advisor 与一名教员建立联系
 - 一名学生最多通过 stud_dept 与一个系关联



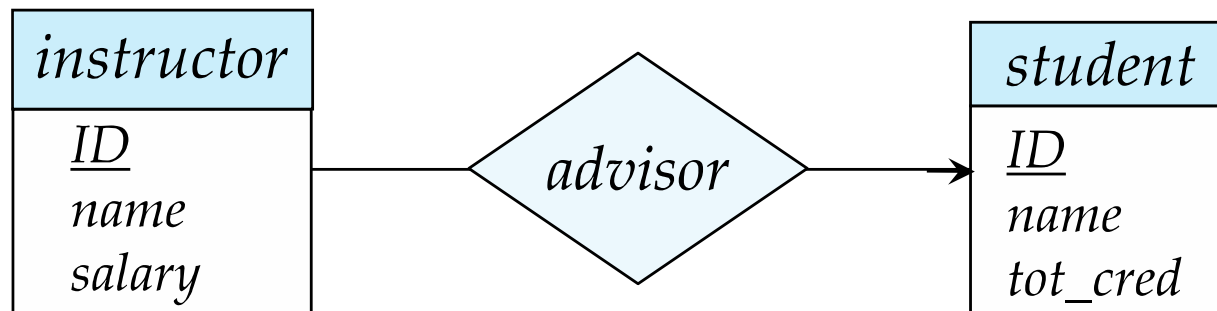
一对多关系

- 教员和学生之间的一对多关系
 - 一名 instructor 通过 advisor 与多名学生（包括 0 名学生）建立了联系
 - 一名学生最多通过 advisor 与一名 instructor 联系，



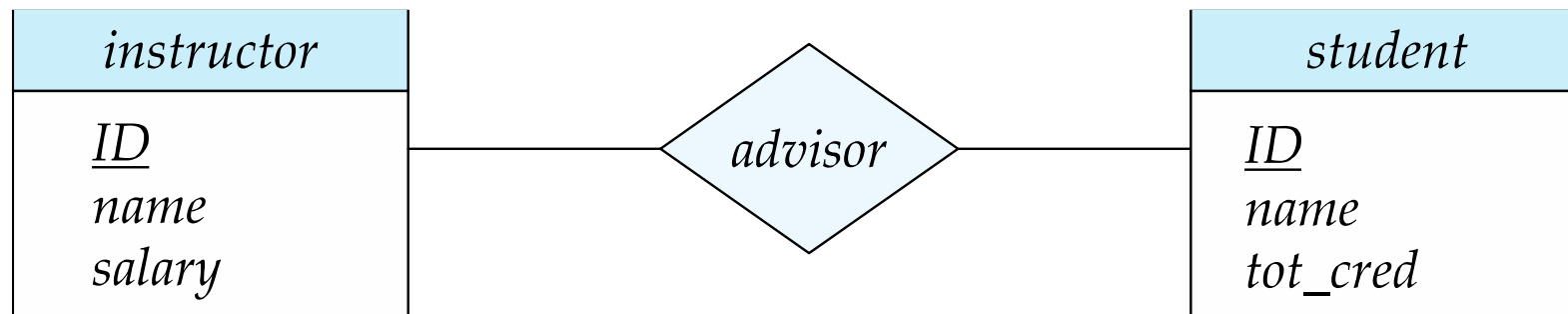
多对一关系

- 在教员和学生的**多对一**关系中，
 - 一名教员最多通过 **advisor** 与一名学生建立联系，
 - 并且一名学生通过 **advisor** 与多位（包括 0 位）教员有联系



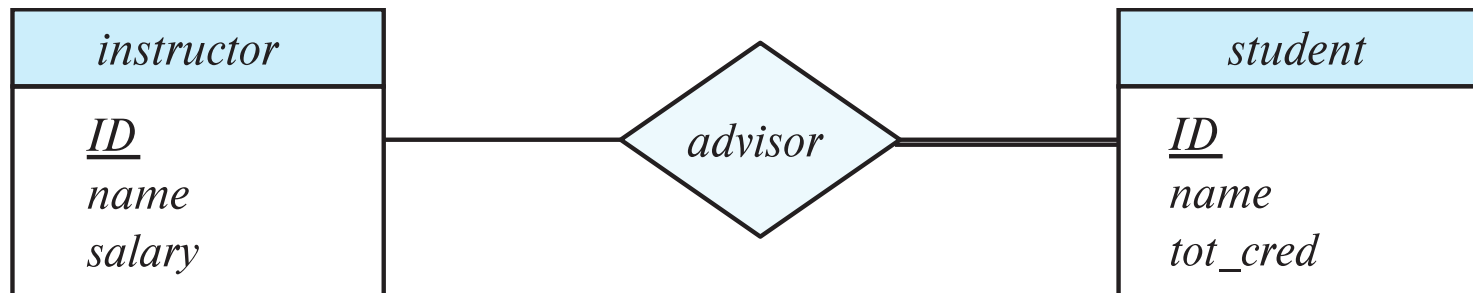
多对多关系

- 一个 教员 与多名（可能为 0 名） 学生 建立联系
- 一个 学生 与多位（可能为 0 位） 导师 建立联系



全部和部分参与

- **全部参与 (Total participation, 用双线表示):**
实体集中的每个实体都参与关系集中的至少一个关系

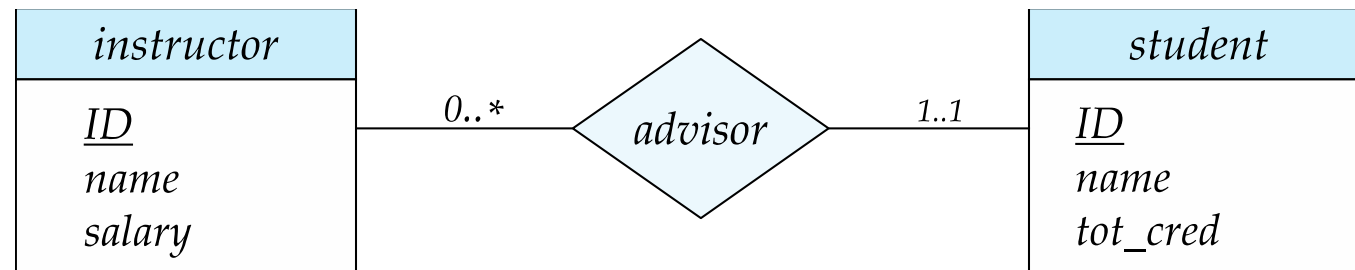


学生参与导师 (advisor) 关系是**全部的**

- 每个学生都必须有一名相关的指导老师
- **部分参与 (Partial participation):** 某些实体可能不参与关系集中的任何关系
 - 例如: 教员 (instructor) 对 导师 (advisor) 的参与是部分的

表达更复杂约束的符号

- 一条线可能有相关的**最小**和**最大基数**，以 $l..h$ 的形式显示，其中 l 是最小基数， h 是最大基数
 - **最小值 1** 表示**全部参与**。
 - **最大值 1** 表示该**实体最多**参与一个关系
 - **最大值 *** 表示**没有限制**。
- 例子



- 导师可以指导 **0** 名或多名学生。一名学生必须有 **1** 名导师，不能有多名导师

三元关系的基数约束

- 允许三元（或更高维度）关系中**最多有一个箭头**来指示基数约束
- **proj_guide** 到 **instructor** 的箭头表示每个学生最多有一个项目指南
- 如果箭头不止一个，则有两种定义其含义的方式。
 - 例如，A、B 和 C 之间的三元关系 R，带有指向 B 和 C 的箭头，可能意味着
 1. 每个 A 实体与 B 和 C 中的一个唯一实体相关联，或者
 2. 来自 (A, B) 的每对实体都与唯一的 C 实体相关联，并且每对 (A, C) 都与唯一的 B 相关联
 - 每种替代方案都有不同的形式
 - 为了避免混淆，我们**禁止使用多支箭**

主键

- **主键**提供了一种指定如何区分实体和关系的方法。我们考虑：
 - 实体集
 - 关系集。
 - 弱实体集

实体集的主键

- 根据定义，各个实体是不同的。
- 从数据库的角度来看，它们之间的差异必须通过它们的属性来表达。
- 实体的属性值的值必须能够唯一地标识该实体。
 - 实体集中不允许有两个实体的所有属性具有完全相同的值。
- **键**是一组足以区分实体的属性

关系集的主键

- 为了区分关系集中的各种关系，我们使用关系集中实体的**单独主键**。
 - 设 R 为涉及实体集 E_1 、 E_2 、.. E_n 的关系集
 - **主键**由实体集 E_1 、 E_2 、.. E_n 的**主键的并集**组成
 - 如果关系集 R 具有与其关联的属性 $a_1, a_2, \dots, a_m$ ，则 R 的主键也包括属性 $a_1, a_2, \dots, a_m$
- 例如：关系集“advisor”。
 - 主键由 *instructor.ID* 和 *student.ID* 组成
- 关系集的主键的选择取决于关系集的映射基数。

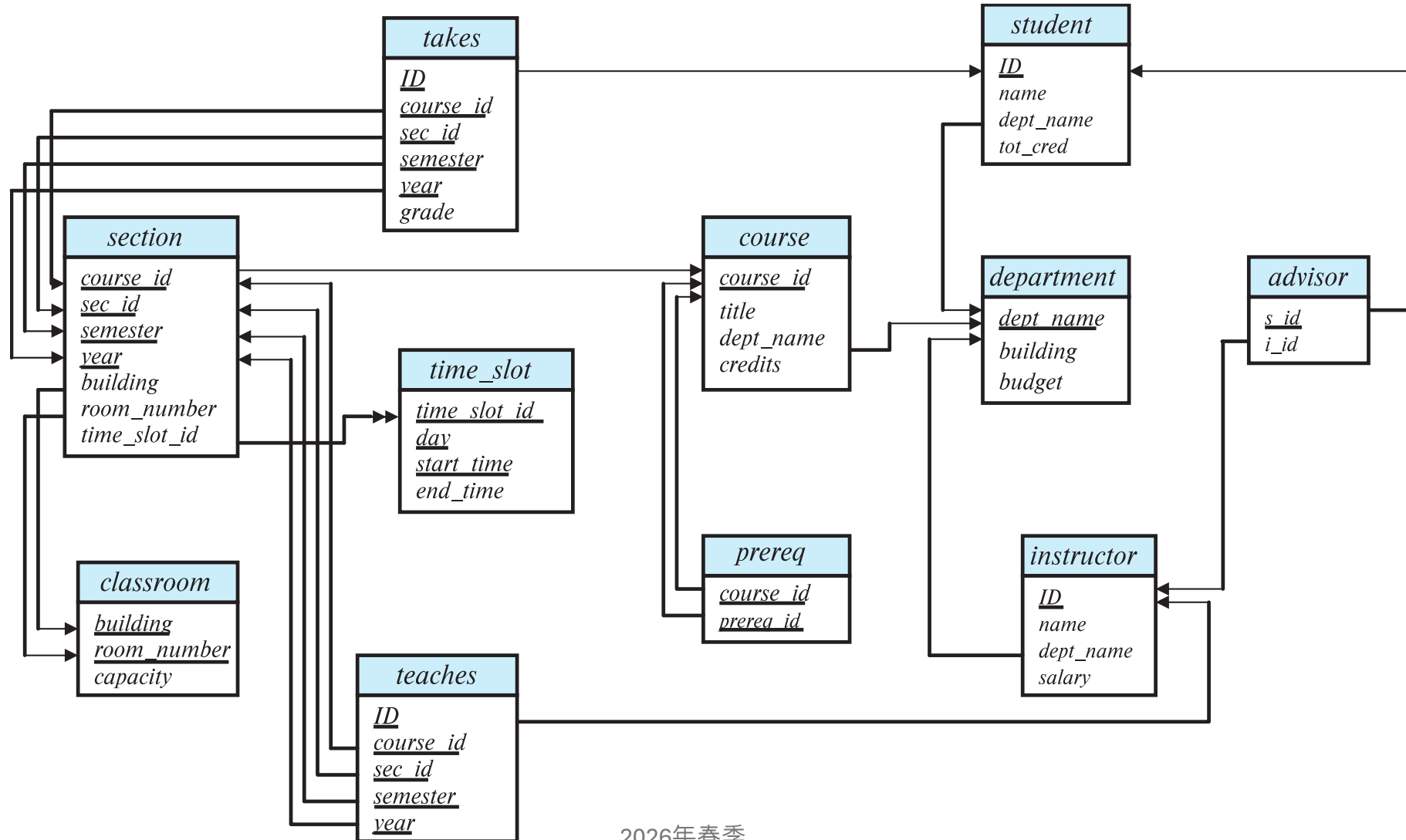
二元关系的主键选择

- 多对多关系。
 - 主键的前述并集是最小超键，并被选为主键。
- 一对多关系。
 - “多”方的主键是一个最小超键，用作主键。
- 多对一关系。
 - “多”方的主键是一个最小超键，用作主键。
- 一对一的关系。
 - 参与实体集的任意一个的主键都形成一个最小超键，可以选择其中一个作为主键。

弱实体集

- 考虑一个 *section* 实体，它由 *course_id*、*semester*、*year* 和 *sec_id* 唯一标识。
- 显然，*section* 实体与 *course* 实体相关。假设我们创建一个关系集 *sec_course* 实体集 *section* 和 *course* 之间。
- *sec_course* 中的信息是多余的，因为 *section* 已经有一个属性 *course_id*，它标识与该 *section* 相关的课程。
- 处理这种冗余的一个选择是摆脱关系 *sec_course*；然而，这样做会使 *section* 和 *course* 之间的关系在属性中变得隐含，这是不可取的。

大学数据库的数据模式



弱实体集 (续)

- 处理这种冗余的另一种方法是**不存储**属性 *course_id*
在 *section* 实体中，仅存储剩余的属性 *section_id*、*year* 和 *semester* 。
 - 然而，实体集 *section* 没有足够的属性来唯一地标识特定的 *section* 实体
- 为了解决这个问题，我们将关系 *sec_course* 作为一种提供额外信息的**特殊关系**，在本例中为 *course_id*，需要唯一地标识 *section* 实体。
- 实体集的存在**依赖于**另一个实体，该实体 (即 另一个实体) 集称为其**标识实体 (identifying entity)**
- 我们不是将主键与弱实体关联，而是使用**标识实体**以及称为**鉴别器 (discriminator)**的**额外属性**来唯一地标识弱实体。

弱实体集 (续)

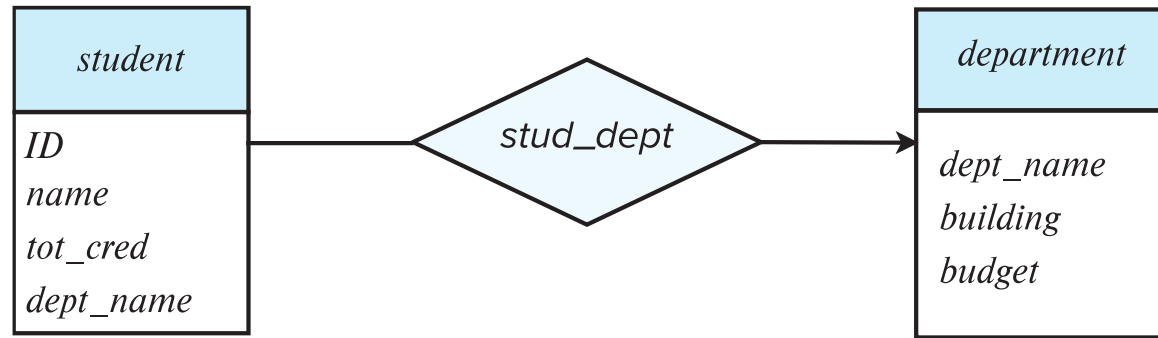
- 不是弱实体集的实体集称为**强实体集**。
- 每个弱实体集都必须与一个**标识实体相关联**；也就是说，弱实体集被称为**依赖于**标识实体集的存在。
- 标识实体集被认为**拥有**它所标识的弱实体集。
- 将弱实体集与标识实体集关联起来的关系称为**标识关系 (identifying relationship)**。
 - 请注意，我们最终从实体集 section 创建的关系模式确实具有属性 `course_id`，原因稍后会清楚，即使我们已经从实体集部分删除了属性 `course_id`。

表达弱实体集

- 在 ER 图中，**弱实体集**通过**双矩形**来表示。
- 我们用**虚线**标出弱实体集的**鉴别器**。
- 集和识别强实体集的关系集用**双菱形**表示。
- ***section*** 的主键 - (***course_id***, ***sec_id***, ***semester***, ***year***)



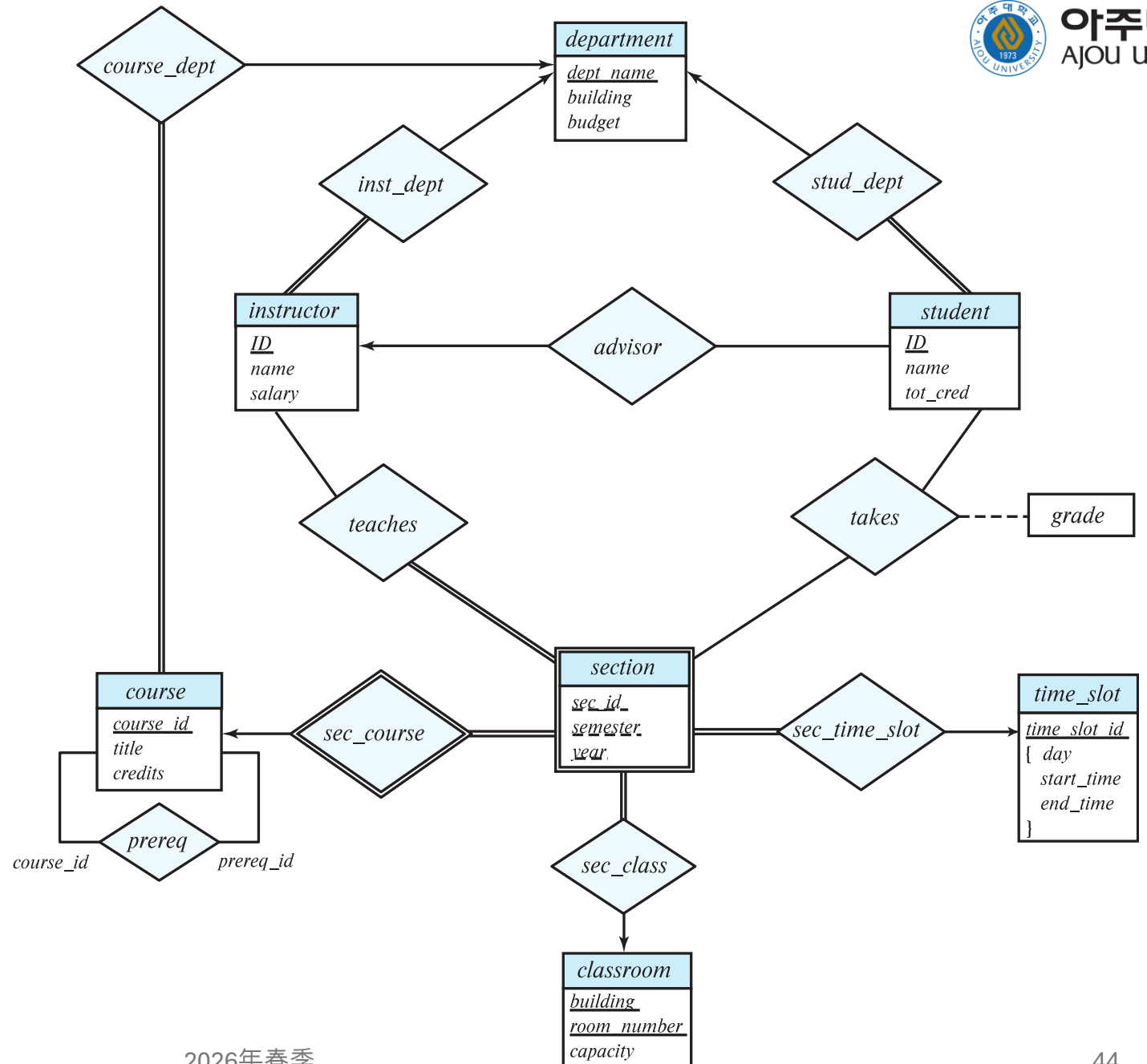
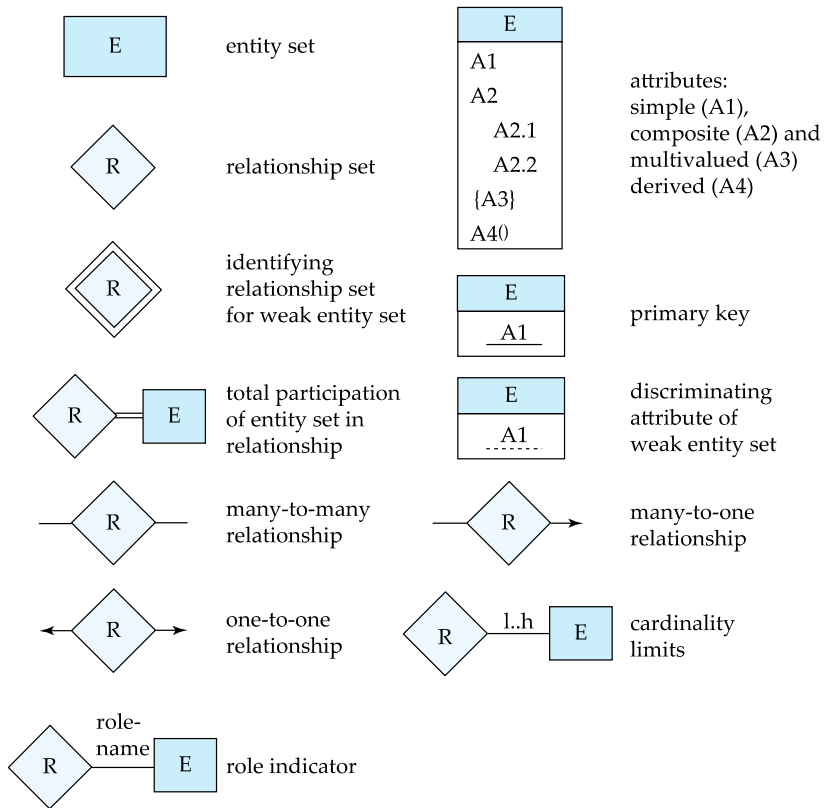
冗余属性



(a) Incorrect use of attribute

- 假设我们有实体集：
 - *student*, 具有属性: *ID*、*name*、*tot_cred*、*dept_name*
 - *department*, 具有属性: *dept_name*、*building*、*budget*
- 我们建模这样一个事实: 使用*stud_dept*关系集, 每一个学生都有一个关联的 *department*。
- *student* 实体下面的属性 *dept_name* 复制了关系中存在的信息, 因此是多余的
 - 并且需要被移除。
- 但是: 当转换回表格时, 在某些情况下该属性会被重新引入, 我们稍后会看到。

大学的ER图



ER模型简化为关系模式

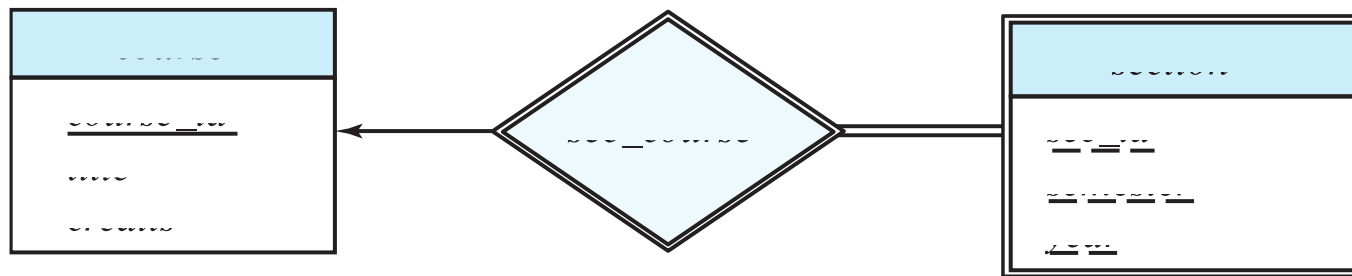
ER Model Reduction to Relation Schemas

简化为关系模式

- 实体集 和 关系集 可以统一表示为代表数据库内容的 **关系模式**。
- 符合 **ER 图** 的数据库可以用 **一组模式来表示**。
- 对于每个实体集和关系集，都有 **一个唯一的模式**，该模式被分配了相应实体集或关系集的名称。
- 每个模式都有许多列（通常对应于属性），这些列具有唯一的名称。

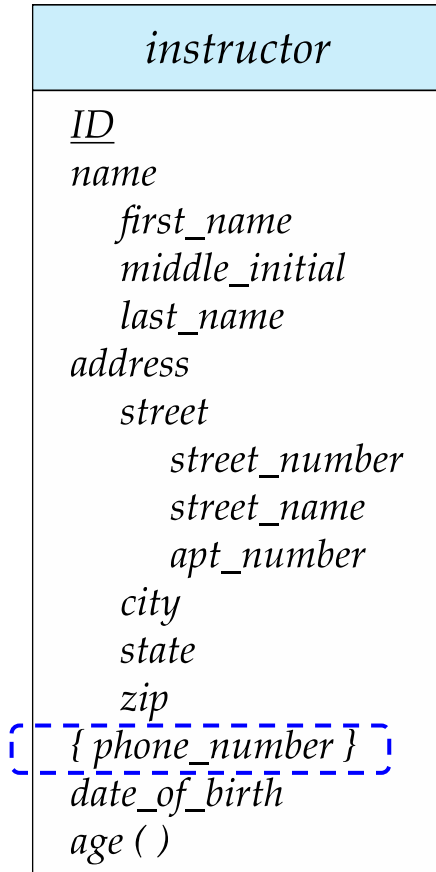
表示实体集

- 实体集简化为具有相同属性的模式
student (ID, name, tot_cred)
- 实体集成为一个表，其中包含用于标识强实体集的主键的列
section (course_ID, semester_ID, semester, year)
- 例子



有复合属性的实体集的表示

- 复合属性创建单独的属性，将复合属性展平
 - 示例: 给定实体集 *instructor*，其复合属性 *name* 包含组件属性 *first_name* 和 *last_name* 对应的实体集的模式有两个属性 *name_first_name* 和 *name_last_name*
 - 如果没有歧义，则省略前缀 (*name_first_name* 可能是 *first_name*)
- 忽略多值属性，扩展的 *instructor* 模式是
 - *instructor*(*ID*, *first_name*, *middle_initial*, *last_name*, *street_number*, *street_name*, *apt_number*, *city*, *state*, *zip_code*, *date_of_birth*)



忽略

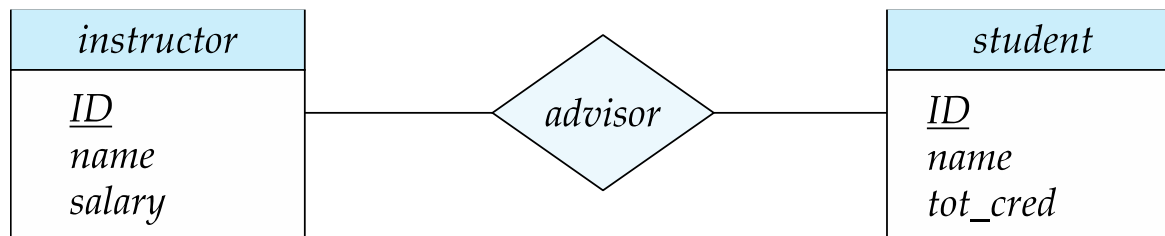
有多值属性的实体集表示

- 实体 E 的多值属性 M 由单独的模式 EM 表示
- 模式 EM 具有与 E 的主键对应的属性，以及与多值属性 M 对应的属性
- 示例：教员的多值属性 *phone_number* 的模式表示如下：
 $inst_phone = (ID, phone_number)$
- 多值属性的每个值映射到模式 EM 上关系的单独元组
 - 例如，主键为 22222、电话号码为 456-7890 和 123-4567 的 instructor 实体映射到两个元组：
(22222, 456-7890) 和 (22222, 123-4567)

关系集 (Relationship Sets) 的表示

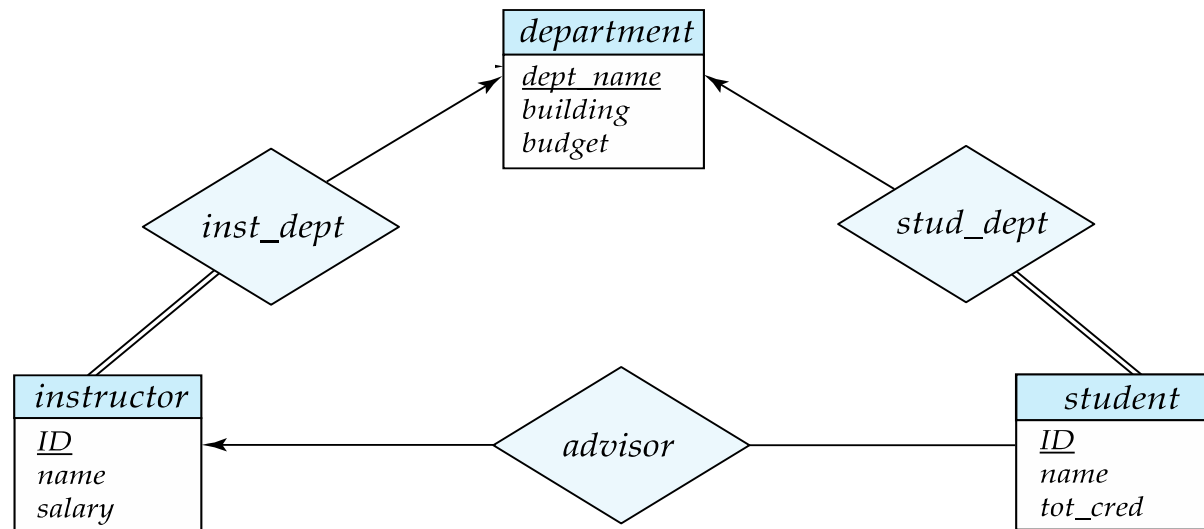
- 多对多关系集表示为一个模式，其中包含两个参与实体集的**主键属性**以及关系集的任何描述性属性。
- *advisor* 的模式

advisor = (*s_id* , *i_id*)



模式的冗余性

- 多对一和一对多关系集在多方是完全的，可以通过在“多”方添加一个额外的属性来表示，该属性包含“一”方的主键
- 例如：不需要创建 *inst_dept* 模式，而是将属性 *dept_name* 添加到由实体集 *instructor* 生成的模式中
- 例子



冗余性

- 对于**一对一**关系集，可以选择任意**一侧**作为“多”侧
 - 也就是说，可以向两个实体集对应的任意一个表中添加一个额外的属性
- 如果“多”方参与是部分的，则用与“多”方对应的模式中的额外属性替换模式可能会导致空值。

模式冗余 (续)

- 将弱实体集 与其 标识强实体集 相链接的 关系集 所对应的模式是冗余的。
- 示例: *section* 架构已包含 *sec_course* 架构中出现的属性



扩展 **ER** 功能

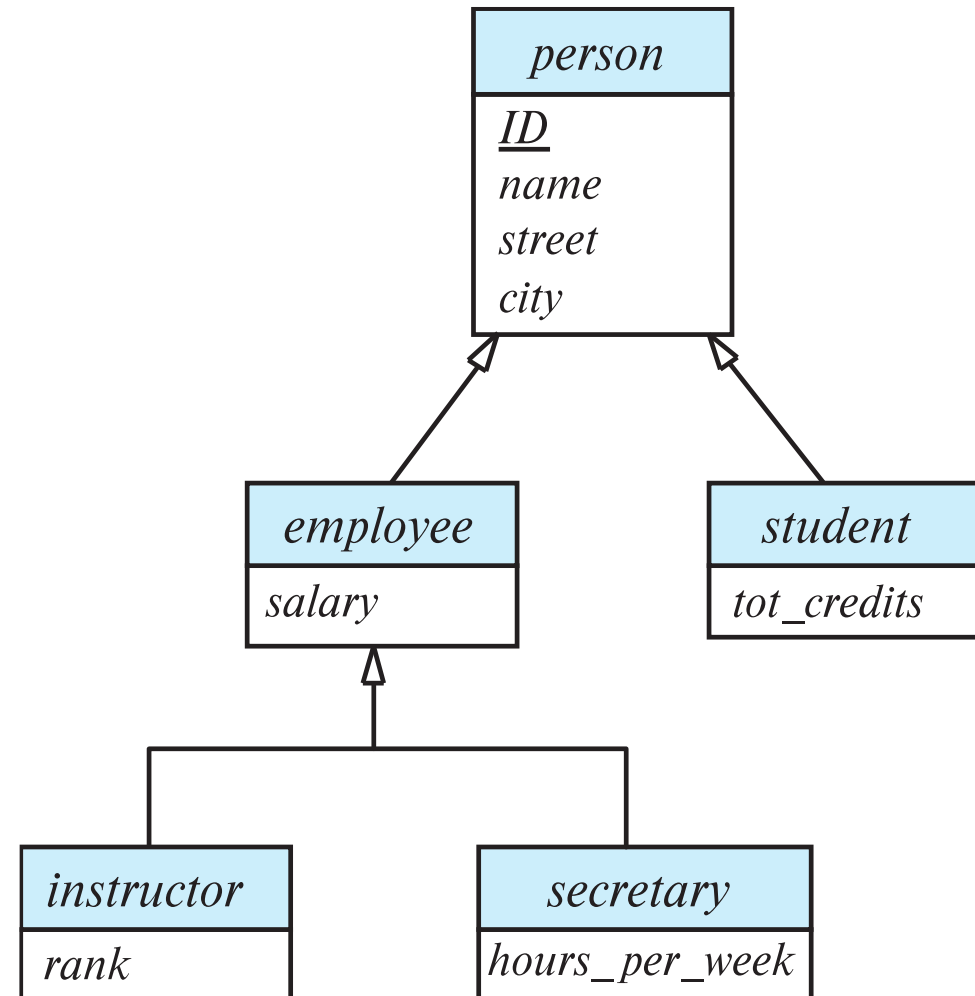
Extended E-R Features

专门化 Specialization

- 自上而下的设计过程；我们指定实体集内的子分组，这些子分组与集合中的其他实体不同。
- 这些子分组成为具有属性或参与不适用于较高级别实体集的关系的较低级别实体集。
- 由一个有ISA (“is a”) 标签的 三角形组件 表示（例如，*instructor* “is a” person）。
- 属性继承 – 较低级别实体集继承了与其链接的较高级别实体集的所有属性和关系参与。

专门化示例

- 重叠 – *employee* 和 *student*
- 不相交 – *instructor* 和 *secretary*
- 全部和部分



通过模式表示专门化

- 方法 1:
 - 更高级别的实体形成模式
 - 为每个低层实体集形成一个模式，包括高层实体集的主键和局部属性

schema	attributes
person	ID, name, street, city
student	ID, tot_cred
employee	ID, salary

- 缺点：获取有关 *employee* 的信息需要访问两个关系，一个对应于低级模式，另一个对应于高级模式

通过模式表示专门化 (续)

- 方法 2:
 - 为每个实体集构建一个包含**所有本地和继承属性的模式**

schema	attributes
person	ID, name, street, city
student	ID, name, street, city, tot_cred
employee	ID, name, street, city, salary

- 缺点：对于同时是 **student** 和 **employee** 的人来说, **name**、**street** 和 **city** 可能会被**重复存储**。

一般化 (泛化) Generalization

- 自下而上的设计过程 - 将具有相同特征的多个实体集组合成更高级别的实体集。
- 专门化和 泛化 是彼此的简单反转；它们以相同的方式在 ER 图中表示。
- 术语 “专门化” 和 “泛化” 可以互换使用。

完整性约束

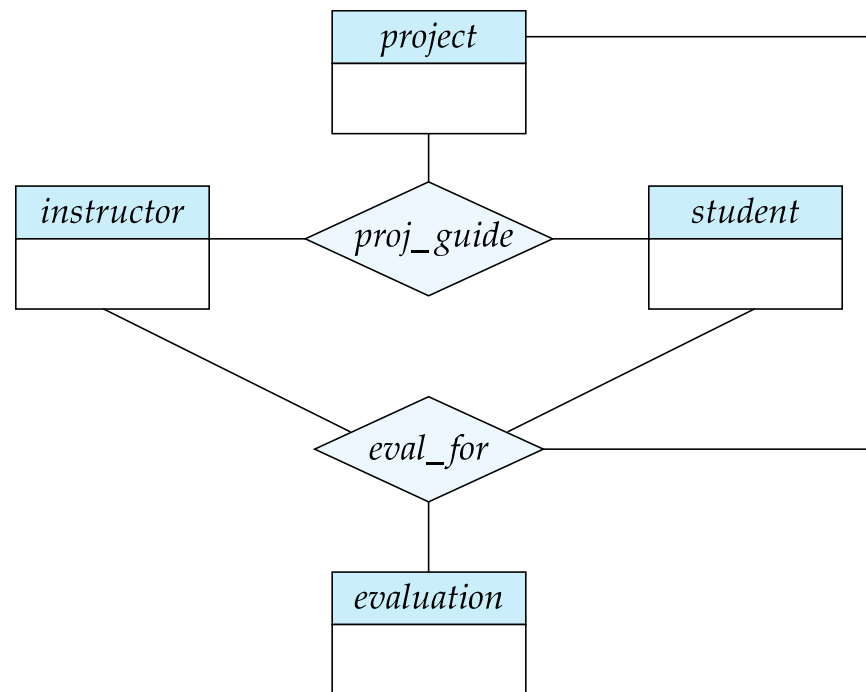
- **完整性约束** - 指定较高级别实体集中的实体是否必须属于泛化中至少一个较低级别实体集。
 - **全部 (total)** : 较高级别实体**必须**属于较低级别实体集之一
 - **部分 (partial)** : 较高级别实体**不需要**属于较低级别的实体集之一

完整性约束 (续)

- 部分泛化是默认设置。
- 我们可以在 ER 图中添加关键字 **total**, 并从关键字到其适用的相应 **空心箭头** (对于完全泛化) 或 到其适用的一组空心箭头 (对于重叠泛化) 绘制一条 **虚线**, 来指定 完全泛化。
- 例如: 学生泛化是完全的:
 - 所有 *student* 实体必须是 研究生 或 本科生。
由于通过泛化得到的高层实体集通常仅由低层实体集中的实体组成, 因此, 泛化的高层实体集的 完备性约束 通常是 完全的

聚合 (Aggregation)

- 我们之前看到的三元关系 *proj_guide*
- 假设我们想记录一个项目 指导老师 对学生的评价

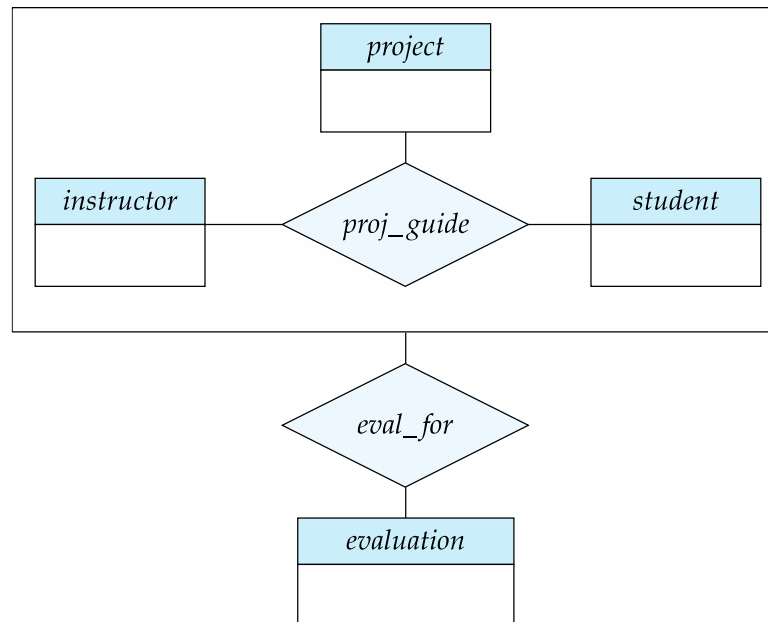


聚合 (续)

- 关系集 *eval_for* 和 *proj_guide* 表示 **重叠** 信息
 - 每个 *eval_for* 关系对应一个 *proj_guide* 关系
 - 然而, 一些 *proj_guide* 关系可能不对应任何 *eval_for* 关系
 - 所以我们不能放弃 *proj_guide* 关系
- 通过 **聚合** 消除这种冗余
 - 将关系视为 **抽象实体 (abstract entity)**
 - 允许 关系之间的 关系
 - 将 关系 抽象为**新实体**

聚合 (续)

- 通过聚合来消除这种冗余，而不引入冗余，下图表示：
 - 学生由特定导师指导特定项目
 - 学生、教师、项目组合可能有相关的评估



简化为关系模式

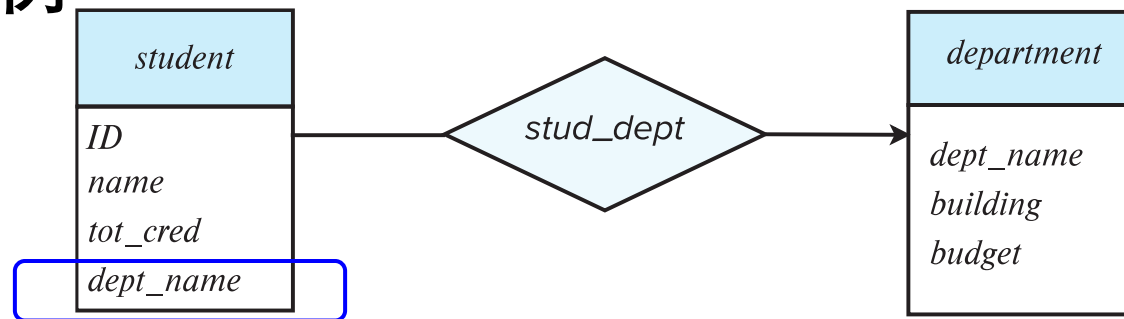
- 为了表示聚合，创建一个包含
 - 聚合关系的**主键**，
 - 关联实体集的**主键**，
 - 任何**描述性属性**。
- 在我们的例子中：
 - 模式 *eval_for* 是：
eval_for (*s_ID*, *project_id*, *i_ID*, *evaluation_id*)
 - 模式 *proj_guide* 是多余的。

设计问题

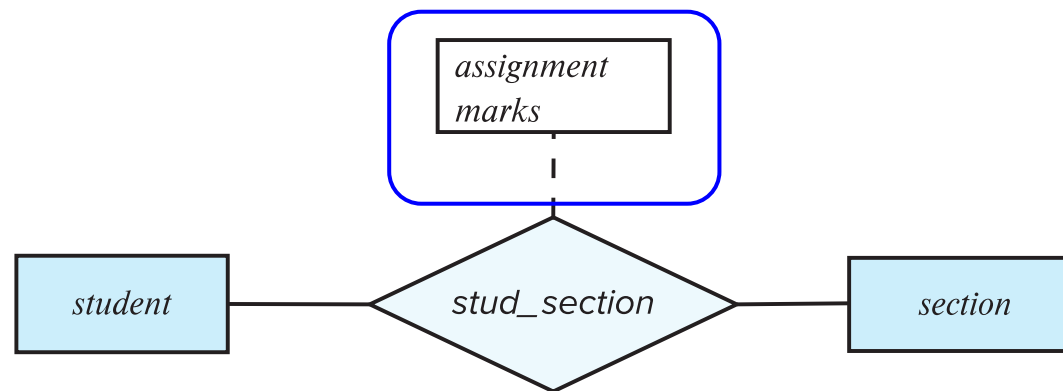
Design Issues

ER图中的常见错误

- 错误 ER 图示例



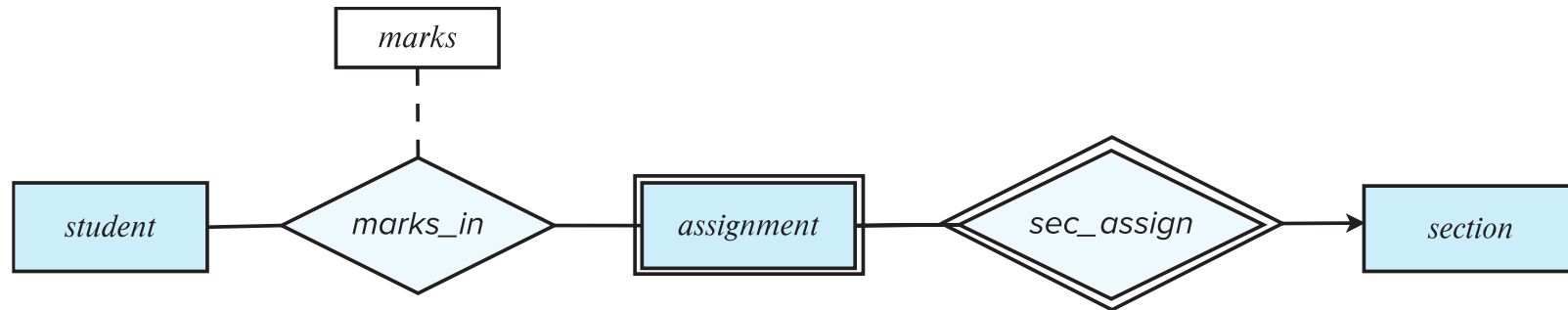
(a) Incorrect use of attribute



(b) Erroneous use of relationship attributes

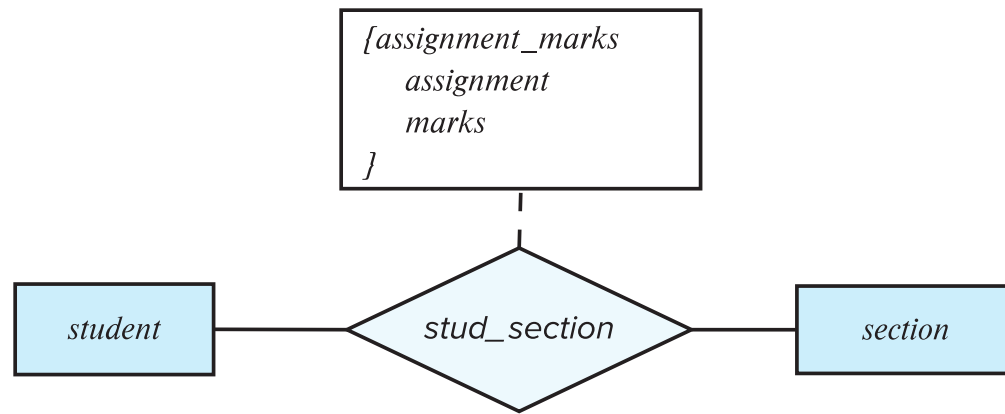
ER 图中常见错误 (续)

正确的设计:



(c) Correct alternative to erroneous E-R diagram (b)

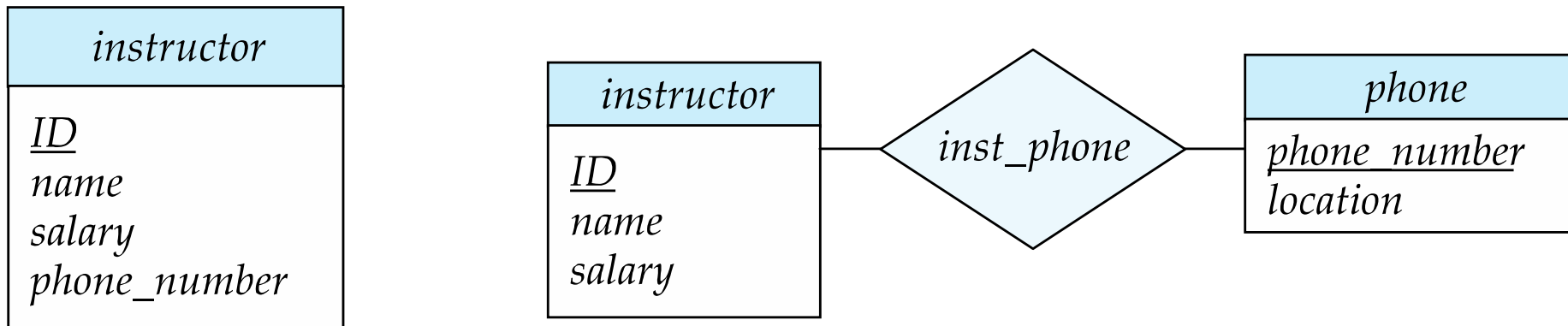
多值 复合属性



(d) Correct alternative to erroneous E-R diagram (b)

实体与属性的使用

- 属性

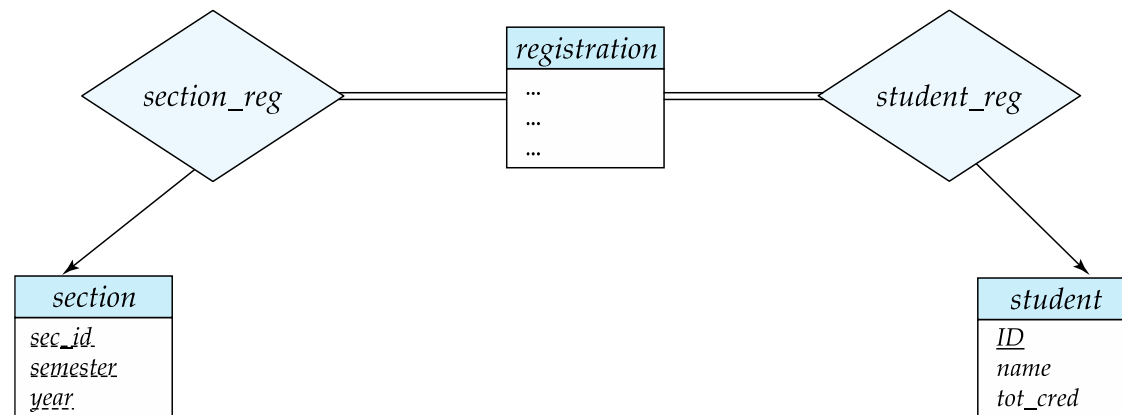


- 使用电话作为实体可以提供有关电话号码的额外信息（以及多个电话号码）

实体与关系集的使用

- 实体集 和 关系集的使用

可能的指导方针是**指定一个关系集**来描述实体之间发生的**动作**



- 关系属性的放置

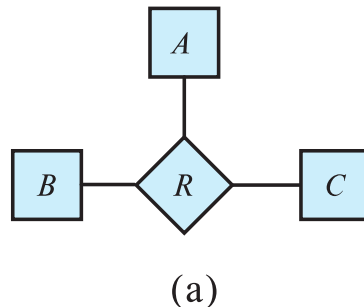
例如, 将日期 (date) 属性作为 advisor 的属性或 student 的属性

二元关系与非二元关系

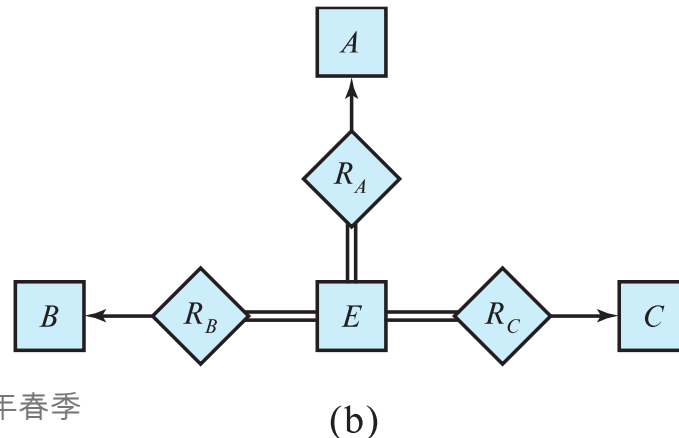
- 尽管可以用多个不同的二元关系集替换任何非二元 (n 元, 其中 $n > 2$) 关系集, 但 n 元关系集更清楚地表明 多个实体 参与 单个关系。
- 一些看似非二元的关系可能用二元关系来更好地表示
 - 例如, 三元关系父母, 即孩子与其父亲和母亲的关系, 最好用两个二元关系 (父亲和母亲) 来代替。
 - 使用两个二元关系可以得到部分信息 (例如, 只知道母亲)
 - 但有些关系天生就不是二元的
 - 示例: proj_guide

将非二元关系转换为二元形式

- 一般来说，任何非二元关系都可以通过创建人造实体集来使用二元关系来表示。
 - 在实体集 A 、 B 和 C 之间替换 R 由一个**实体集 E** 和三个关系集组成：
 - R_A ，关联 E 和 A
 - R_B ，关联 E 和 B
 - R_C ，关联 E 和 C
 - E 创建标识属性并将 R 的任何属性添加到 E
 - R 中的每个关系 (a_i, b_i, c_i) ，创建
 - 一个新的实体 e_i 在实体集 E
 - 将 (e_i, a_i) 添加到 R_A
 - 添加 (e_i, b_i) 至 R_B
 - 将 (e_i, c_i) 添加到 R_C



(a)



(b)

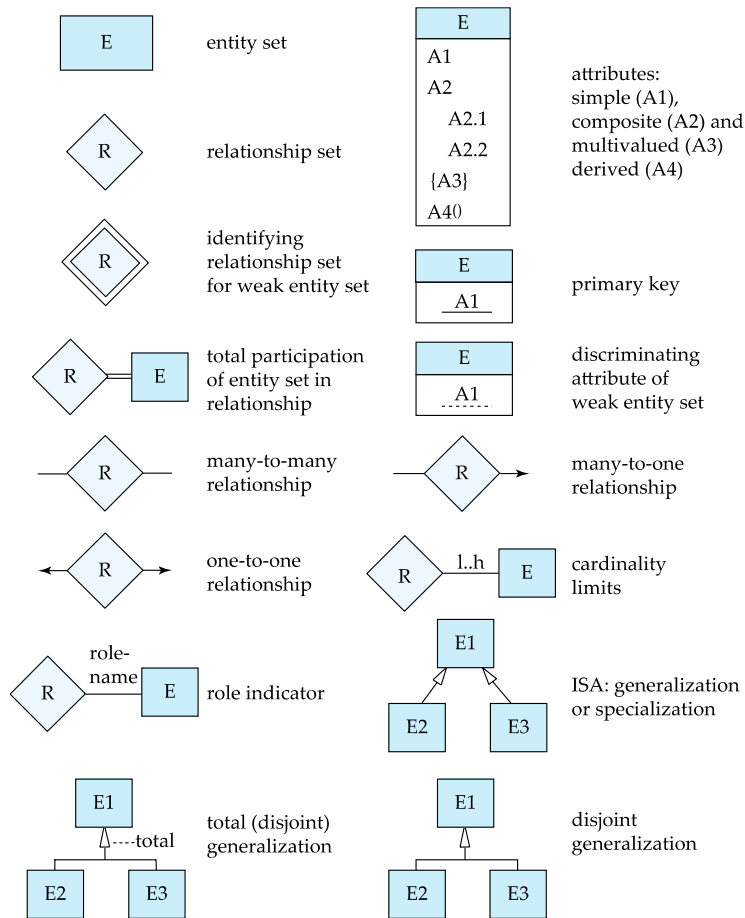
转换非二元关系 (续)

- 还需要转换**约束**
 - 可能无法实现转换所有约束
 - 转换后的模式中可能存在无法与 R 的任何实例对应的实例
 - 练习：向关系 R_A 、 R_B 和 R_C 添加约束，以确保新创建的实体与实体集 A 、 B 和 C 中各有一个实体相对应
 - 通过使 E 成为一个由三个关系集 标识的 **弱实体集** 来避免创建标识属性

ER 设计决策

- 使用 属性 或 实体集 来表示 对象。
- 现实世界的概念最好用 实体集 或 关系集 来表达。
- 使用 三元关系 或 一对二元关系。
- 使用 强 或 弱实体集。
- 使用 专门化/泛化 - 有助于设计的模块化。
- 聚合的使用 - 可以将 聚合实体集 视为单个单元，而无需关心其内部结构的细节。

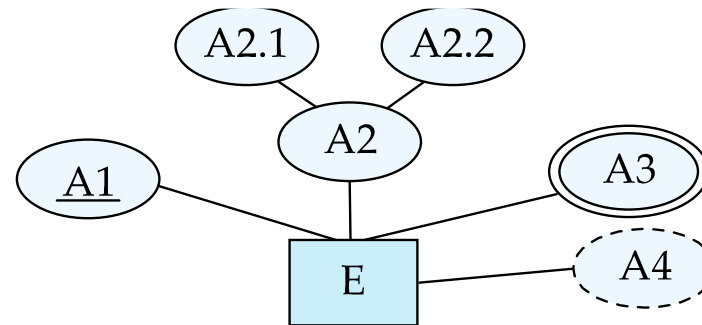
ER 符号表中使用的符号摘要



其他类型的 ER 符号定义

- **Chen, IDE1FX,**

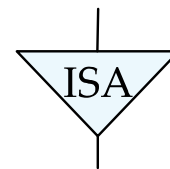
entity set E with
simple attribute A1,
composite attribute A2,
multivalued attribute A3,
derived attribute A4,
and primary key A1



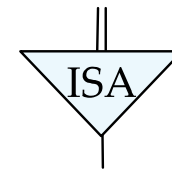
weak entity set



generalization



total
generalization

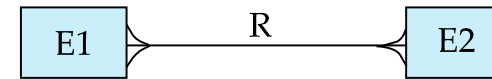
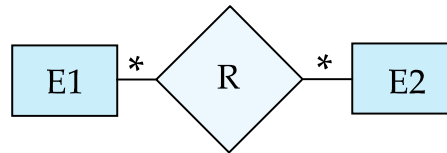


其他类型的 ER 符号定义

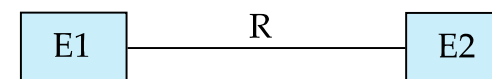
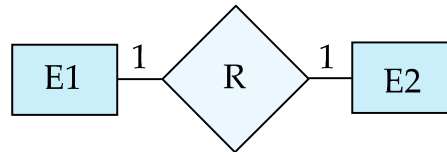
Chen

IDE1FX (鱼尾纹符号) (Crows feet notation)

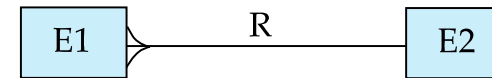
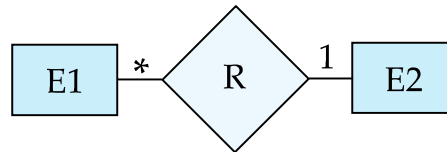
many-to-many
relationship



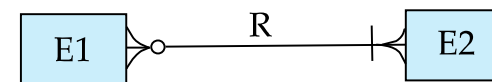
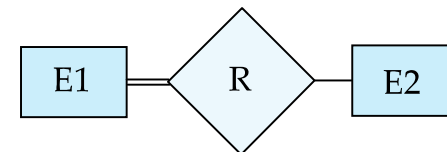
one-to-one
relationship



many-to-one
relationship



participation
in R: total (E1)
and partial (E2)

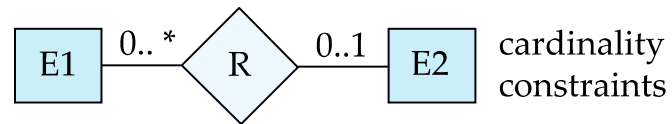
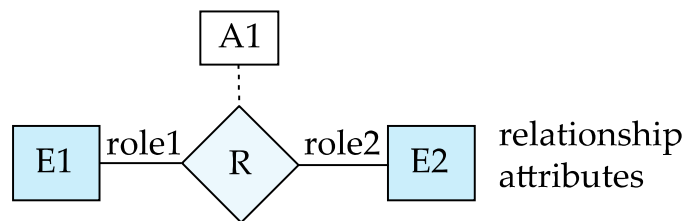
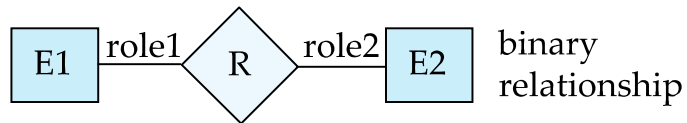
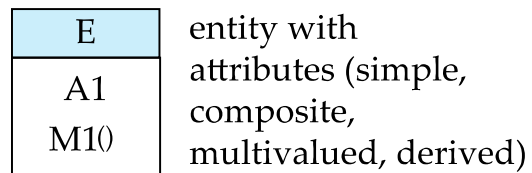


UML, 统一建模语言

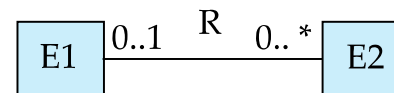
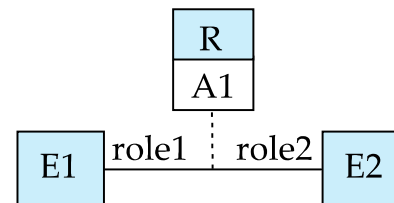
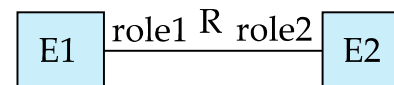
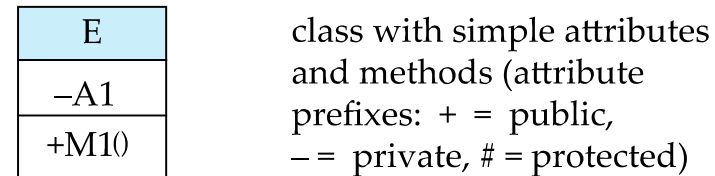
- **UML** : Unified Modeling Language
- UML 有许多组件, 可以图形化地模拟整个软件系统的不同方面
- UML 类图 (class diagrams) 与 ER 图相对应, 但有几个区别。

ER 与 UML 类图

ER Diagram Notation



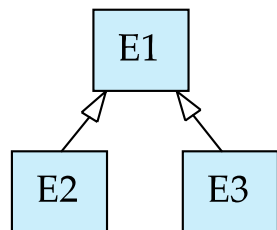
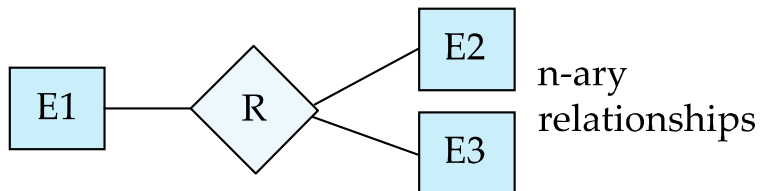
Equivalent in UML



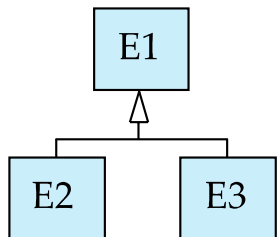
* 注意基数约束描述中的位置反转

ER 与 UML 类图

ER 图符号

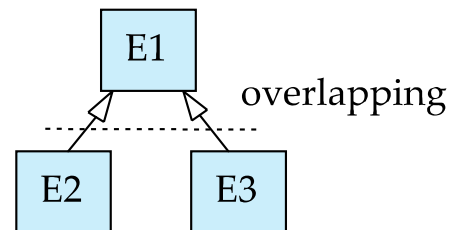
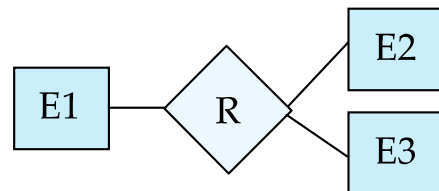


overlapping
generalization

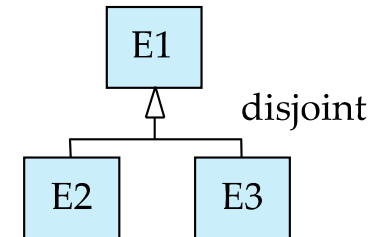


disjoint
generalization

UML 中的等效项



overlapping



disjoint

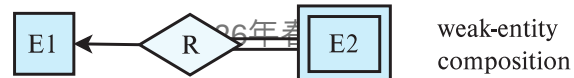
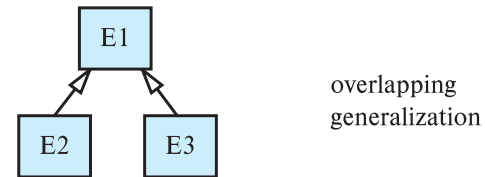
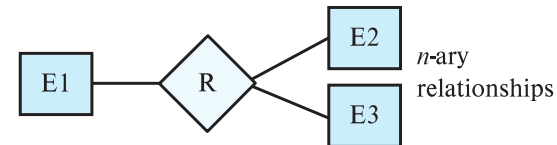
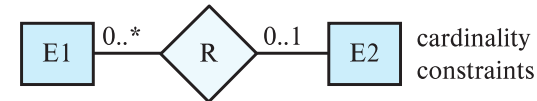
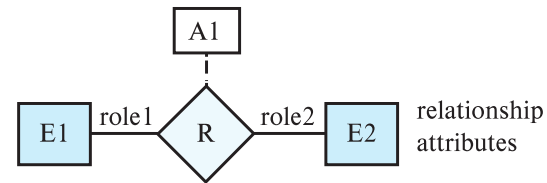
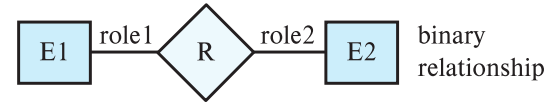
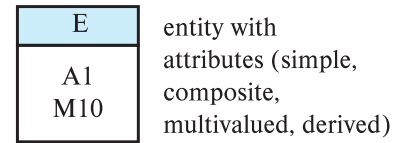
* 泛化可以使用 合并 或 单独 的箭头独立于 disjoint/overlap

UML 类图 (续)

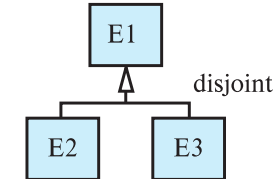
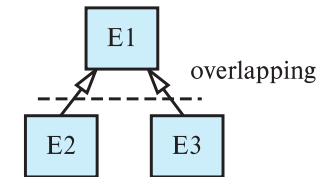
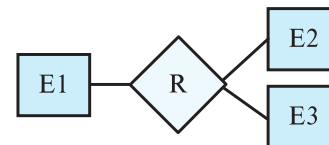
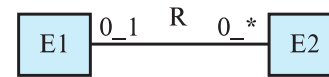
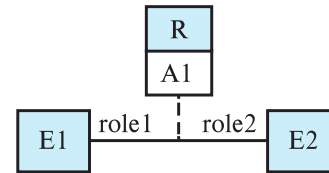
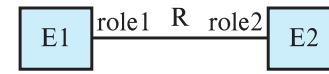
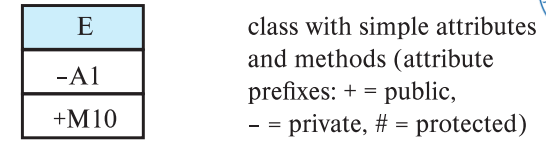
- 在 UML 中，二元关系集的表示方式是，只需画一条连接实体集的线。关系集的名称写在线的旁边。
- 实体集在关系集中扮演的角色也可以通过在实体集旁边的线上写上角色名称来指定。
- 或者，可以将关系集名称与关系集的属性一起写在一个框中，然后使用虚线将该框连接到描述关系集的线。

ER 与 UML 类图

ER Diagram Notation



Equivalent in UML



数据库设计的其他方面

- 功能要求
- 数据流、工作流
- 模式演化

Database System

Thank you! Q/A?

Instructor: Yiwen Shen, Ph.D.

Dept. of Software & Computer Engineering,

Ajou University, Korea

chrisshen@ajou.ac.kr